

nyc-uoiP/ — Repository Structure

NYC Urban Operations Intelligence Platform · Monorepo layout for data engineering teams

ROOT

nyc-uoiP/ ← repo root

- ├─ **README.md** project overview, quickstart, architecture diagram link
- ├─ **CLAUDE.md** Claude Code agent instructions
- ├─ **AGENTS.md** all AI agent tool conventions
- ├─ **datacontract.yaml** top-level data contract entry (Open Data Contract Standard)
- ├─ **pyproject.toml** unified Python dep management (uv / Poetry)
- ├─ **Makefile** common dev commands: lint, test, deploy-dags, spark-submit
- └─ **.env.example** env var template — never commit .env

INGESTION — AIRFLOW DAGS & OPERATORS

- ├─ **dags/** **Phase 1 + 2**
 - ├─ **dag_ingest_nyc_311.py** incremental pull, pagination, GCS/MinIO Bronze write
 - ├─ **dag_ingest_nypd_collisions.py** 7-day lookback window for late-arriving facts
 - ├─ **dag_ingest_open_meteo.py** daily snapshot, 7-day forecast + 3-day history
 - ├─ **dag_ingest_borough_boundaries.py** one-time static GeoJSON load
 - ├─ **dag_etl_bronze_to_silver.py** triggers Spark job after ingestion completes
 - ├─ **dag_etl_silver_to_gold.py** BigQuery / Trino Gold layer load
 - ├─ **dag_intelligence_engine.py** daily load score + recommendation calculation
 - └─ **operators/** custom Airflow operators & sensors
 - ├─ **socrata_to_gcs_operator.py**
 - └─ **dataproc_spark_operator.py**

INGESTION — PYTHON API CLIENTS

- ├─ **ingestion/**
 - ├─ **clients/** thin API wrappers, one file per source
 - ├─ **socrata_client.py** pagination, App Token, rate-limit retry
 - ├─ **open_meteo_client.py**
 - └─ **nyc_open_data_client.py** GeoJSON static download
 - ├─ **loaders/** write raw files to Bronze storage
 - ├─ **gcs_loader.py** **Phase 1**
 - └─ **minio_loader.py** **Phase 2**
 - └─ **schemas/** Pydantic models — validate raw API response shape
 - ├─ **nyc_311_raw_schema.py**
 - ├─ **nypd_collisions_raw_schema.py**
 - └─ **open_meteo_raw_schema.py**

SPARK ETL — BRONZE → SILVER

```

└─ spark/
  └─ jobs/ one PySpark entry-point per dataset
    └─ etl_nyc_311.py
    └─ etl_nypd_collisions.py
    └─ etl_open_meteo.py
  └─ transforms/ reusable transform functions, importable by jobs
    └─ deduplication.py hash-key dedup + Iceberg MERGE INTO (Phase 2)
    └─ timestamp_normalizer.py EST → UTC, string → timestamp
    └─ geo_enrichment.py borough fill via zip lookup + ST_CONTAINS
    └─ complaint_standardizer.py lower-case, strip, category mapping
  └─ schemas/ Silver layer StructType definitions
    └─ silver_311_schema.py
    └─ silver_collisions_schema.py
    └─ silver_weather_schema.py
  └─ quality/ data quality checks (Great Expectations / Deequ)
    └─ expectations_311.json
    └─ deequ_checks_collisions.py Phase 2

```

WAREHOUSE — SQL (BIGQUERY / TRINO)

```

└─ sql/
  └─ ddl/ CREATE TABLE statements, run once at setup
    └─ bigquery/ Phase 1
      └─ fact_311_requests.sql
      └─ fact_vehicle_collisions.sql
      └─ dim_date.sql
      └─ dim_geography.sql GEOGRAPHY type, ST_GEOGPOINT
      └─ dim_weather_forecast.sql
    └─ iceberg/ Phase 2 Iceberg DDL via Trino / Spark SQL
  └─ dml/ daily incremental loads, MERGE / INSERT OVERWRITE
    └─ load_fact_311.sql
    └─ load_fact_collisions.sql
    └─ spatial_borough_fill.sql ST_CONTAINS join to back-fill borough_id
  └─ intelligence/ operational scoring + recommendation rules
    └─ calc_load_score.sql  $0.4 \times 311 + 0.4 \times \text{collision} + 0.2 \times \text{weather}$ 
    └─ calc_operational_drivers.sql
    └─ calc_resource_recommendations.sql
    └─ populate_daily_summary.sql writes to fact_daily_operational_summary

```

INFRASTRUCTURE AS CODE

```

└─ infra/
  └─ terraform/ Phase 1 — GCP
    └─ main.tf GCS buckets, BigQuery datasets, IAM, Composer env
    └─ variables.tf
    └─ outputs.tf

```

```
| └─ docker/   Phase 2 — self-hosted
|     └─ docker-compose.yml  Airflow + Spark + Trino + MinIO + Hive Metastore
|     └─ airflow.Dockerfile
|     └─ spark.Dockerfile
```

DATA CONTRACTS & SOURCE REGISTRY

```
└─ contracts/
|   └─ source-registry.md  SRC-NYC-001 ~ 004 authority list
|   └─ api-contracts/   JSON Schema for each upstream API response
|       └─ nyc-311-schema.json
|       └─ nypd-collisions-schema.json
|       └─ open-meteo-schema.json
|   └─ consumer-contracts/  what downstream (Gold / dashboard) expects
|       └─ gold-layer-contract.yaml
```

TESTS

```
└─ tests/
|   └─ unit/  pure Python, no Spark cluster needed
|       └─ test_socrata_client.py
|       └─ test_timestamp_normalizer.py
|       └─ test_deduplication.py
|   └─ integration/  requires local Spark / BigQuery sandbox
|       └─ test_etl_nyc_311_pipeline.py
|       └─ test_dag_end_to_end.py
|   └─ fixtures/  small sample JSON files, mock API responses
|       └─ sample_311_response.json
|       └─ sample_collision_response.json
```

CI/CD & TOOLING

```
└─ .github/
|   └─ workflows/
|       └─ ci.yml  lint (ruff) + unit tests on every PR
|       └─ deploy-dags.yml  push DAGs to GCS / Composer on merge to main
|       └─ copilot-instructions.md
└─ .pre-commit-config.yaml  ruff, sqlfluff, conventional commits
└─ pyproject.toml  ruff + pytest config unified here
```

Phase 1 GCP Demo (BigQuery, Dataproc, GCS, Composer)

Phase 2 Self-built cluster (MinIO, Iceberg, Trino, Docker)

Phase 1+2 Shared across both phases